

COMP364 Quiz 4

Question 1. Computing convolution output values

Each of the following questions describes two adjacent layers in a one-dimensional convolutional neural network: upstream layer A and downstream layer B . In each case, give the values computed at B based on inputs from A . Figure 1 summarizes the relevant channel values and kernels used in each part of the question.

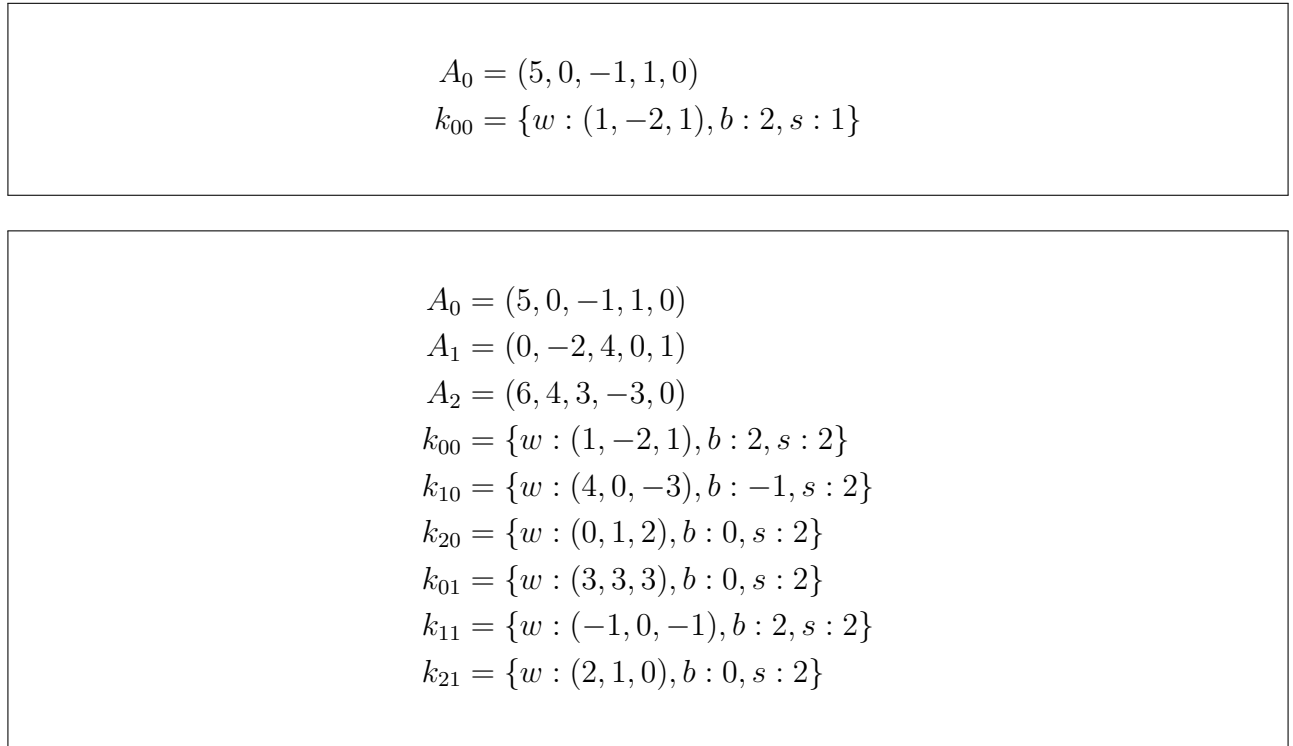


Figure 1: **Top:** Channel value and kernel for question 1(a). **Bottom:** Channel values and kernels for questions 1(b)–(e).

(a) **(5 points)** Layer A has one channel, A_0 , of length 5. Its output is $A_0 = (5, 0, -1, 1, 0)$. B has 1 channel, B_0 . Channel B_0 is produced from channel A_0 by cross-correlating with a kernel of size 3 and stride 1; the kernel is $(1, -2, 1)$ with bias 2, written

$$k_{00} = \{w : (1, -2, 1), b : 2, s : 1\}.$$

More generally, the notation k_{ij} denotes the kernel mapping channel A_i to channel B_j . The symbol w designates the weights, b designates the bias, and s designates the stride. We use the standard CNN convention here, so the kernel is not flipped.

(b) **(5 points)** Everything is identical to the previous question except that the kernel stride is 2.

	channels in A	length of A	kernel width	kernel stride	padding (at each end)	bias	channels in B	length of B	num parameters in B	num ignored inputs
(a)	1	64	8	1	0	False	1			
(b)						True				
(c)	6									
(d)							10			
(e)						False				
(f)				2						
(g)			7	1						
(h)				2						
(i)					1					
(j)					2					
(k)					3					

Figure 2: Details for question 2. Fill in the last 3 columns.

(c) **(5 points)** Everything is identical to (b) except that A now has a second channel with outputs $A_1 = (0, -2, 4, 0, 1)$, and B has a corresponding new kernel $k_{10} = \{w : (4, 0, -3), b : -1, s : 2\}$.

(d) **(5 points)** The same as (c) except now we add a third channel to A : this is channel A_2 with outputs $A_2 = (6, 4, 3, -3, 0)$ and kernel $k_{20} = (0, 1, 2; 0)$.

(e) **(10 points)** The same as (d) except now we add a second channel to B , so A has three channels with IDs 0, 1, 2; and B has two channels with IDs 0, 1. The new kernels for channel B_1 are

$$\begin{aligned}
 k_{01} &= \{w : (3, 3, 3), b : 0, s : 2\} \\
 k_{11} &= \{w : (-1, 0, -1), b : 2, s : 2\} \\
 k_{21} &= \{w : (2, 1, 0), b : 0, s : 2\}
 \end{aligned}$$

Question 2. Computing convolution output sizes (33 points)

Figure 2 on page 2 describes various configurations for two consecutive layers, A and B , in a one-dimensional convnet. Each row of the table shows only the *changes* in configuration values compared to the previous row. So you can assume that all other configuration values remain the same as the previous row. For example, configuration (b) is the same as (a) except that (b) employs biases in its units. The changes are cumulative. For example, (c) is the same as (b) except that (c) has 6 channels, and (c) *does* employ biases, as this property is inherited from (b).

The task for this question is to fill in the last three columns of figure 2 on page 2.

Question 3. Computing convolution output size with additional layer (10 points)

Consider a convolutional network that has layers A and B exactly as in the final part of the previous question, 2(k). We add another layer C , which has 5 channels and employs a kernel of

```

1 import torch
2 import torch.nn as nn
3 loss_fn = nn.MSELoss()
4 target = torch.tensor([[1, 2], [20, 21]], dtype=torch.float32)
5 input = torch.tensor([[1.1, 2], [20, 20]], dtype=torch.float32, requires_grad=True)
6 num_vals = input.numel()
7 print(f"num_vals: {num_vals}")
8 loss = loss_fn(input, target)
9 loss_val = loss.item()
10 print(f"loss_val*num_vals: {loss_val * num_vals}")
11 print(f"loss_val: {loss_val}")
12 loss.backward()
13 print(f"dLoss/dInput * num_vals: \n{input.grad * num_vals}")
14 print(f"dLoss/dInput: \n{input.grad}")
15 optimizer = torch.optim.SGD([input], lr=0.1)
16 optimizer.step()
17 print(f"Updated input: \n{input.detach()}")

```

Figure 3: PyTorch code for question 4

width 7, stride 2, no bias, and padding 3 on each side. Give the (a) shape and (b) parameter count of layer C .

Question 4. Regression using MSE and PyTorch

(a) **(2 points)** Given output values $x_1, x_2, \dots, x_n$ and target values $t_1, t_2, \dots, t_n$, what function is minimized when using mean squared error (MSE)?

(b) **(15 points)** Figure 3 shows some code for performing regression in PyTorch using mean squared error (MSE), and figure 4 shows the output of this code. Give a brief explanation of each line of output.

Total number of points: **90**

```
1 num_vals: 4
2 loss_val*num_vals: 1.0099999904632568
3 loss_val: 0.2524999976158142
4 dLoss/dInput * num_vals:
5 tensor([[ 0.2000,  0.0000],
6         [ 0.0000, -2.0000]])
7 dLoss/dInput:
8 tensor([[ 0.0500,  0.0000],
9         [ 0.0000, -0.5000]])
10 Updated input:
11 tensor([[ 1.0950,  2.0000],
12         [20.0000, 20.0500]])
```

Figure 4: Output of code in figure 3